

**Project Name:** Spacecrypt

**Team Name:** Spacecrypt

**Team:**

- Sanil Katula (Team Lead) – [sanilkatula@ucsb.edu](mailto:sanilkatula@ucsb.edu),
- Yarwin Liu (Scribe) – [yarwinliu@ucsb.edu](mailto:yarwinliu@ucsb.edu),
- Aryan Chopra – [aryanchopra@ucsb.edu](mailto:aryanchopra@ucsb.edu),
- Shashank Bhagwani – [shashank790@ucsb.edu](mailto:shashank790@ucsb.edu),
- Karthik Bhattaram – [kbhattaram@ucsb.edu](mailto:kbhattaram@ucsb.edu),

**Mentors:**

- Filip Rezabek – [filip@spacecomputer.io](mailto:filip@spacecomputer.io),
- Amir Yahalom – [amir@spacecomputer.io](mailto:amir@spacecomputer.io),
- Eason Chai – [eason@spacecomputer.io](mailto:eason@spacecomputer.io),

## Introduction

### **Project Overview**

Our team's vision is to push the boundaries of how security and location-awareness intersect in everyday digital experiences. We believe that presence should not only be provable, but also trustworthy and usable in ways that unlock creativity, fairness, and safety. By combining proof-of-presence with cryptographically strong randomness, we aim to create a new class of location-aware secure applications.

### **Background, Significance, and Strategic Fit**

Various applications - from testing software to Pokémon Go - rely on verifying user location. If bypassed, students could take exams from their home, and gamers could cheat by capturing Pokémon from afar. However, by combining the location tracking with cryptography, we can theoretically ensure that locations are true.

Expected benefits include proof of location that cannot be spoofed and a truly random number that cannot be guessed.

### **Digital Signatures**

The security triad has three components - confidentiality, integrity, and availability, and a digital signature guarantees the first two. It works by creating a hash, or 'summary' of the text, and encrypting this with the private key. The message itself is often encrypted as well. A key focus of our project is accessing location data, and digital signatures would be instrumental in safeguarding this information. A MITM, or man in the middle attack, entails capturing data in

transit, modifying it, and racing against the sender to deliver a corrupted version of the message to the intended recipient. Now with a digital signature, the hash appended to the message will not match the corrupted version. A raspberry PI might be responsible for verifying the user's location.

## **Cryptographically Strong Random Number Generators**

Pseudorandom number generators are easy to implement and calculate, but have the weakness of producing 'random' numbers in what is really a predictable sequence. If used in a game, this could mean knowing what previous die rolls are means knowing with certainty what the next one will be, which would completely derail the user experience. Now in truly random number generators, the next generated bit is independent of what has been produced previously, and one method of obtaining these is using cosmic radiation. This method is used by Cloudflare, but our system is tamper proof as it is immune to damaged fiber optic cables, earthquakes, and geopolitical incidents.

## **Problem Statement**

Current location-aware applications (e.g., navigation, gaming, secure access control) rely on inadequate trust models. IP address based geolocation often lacks precision, and addresses can be easily spoofed with a proxy. An example of this would be ensuring that test takers are where they claim to be - if someone were to take an exam from their home, cheating might ensue. At the same time, many digital systems depend on weak randomness sources (e.g., pseudo-random generators), which undermines fairness and security in lotteries, raffles, authentication, and key generation. If someone had access to an older encryption key, they might be able to predict the next one. Without a verifiable presence mechanism and a cryptographically auditable randomness source, applications remain vulnerable to fraud, spoofing, replay attacks, and unfair outcomes.

## **Suggested Approach**

We propose to integrate Proof-of-Presence (PoP) with a cryptographically strong True Random Number Generator (cTRNG) into a unified platform:

**Proof-of-Presence:** Prototype using Raspberry Pi beacons as broadcast devices, combined with a React Native mobile app capable of fusing Bluetooth, WiFi, and GPS signals. Presence will be verified via signal strength thresholds, timestamps, and signed cryptographic challenges to mitigate spoofing.

**cTRNG Service:** Build a REST API backed by hardware/OS entropy sources, providing unpredictable and auditable randomness. Commit-reveal protocols and digital signatures will ensure results are verifiable by anyone, preventing tampering.

**Integration Layer & Demonstrations:** Combine PoP and cTRNG to create presence-locked, verifiably fair applications such as:

- Raffles and loot drops are limited to users inside a beacon zone.

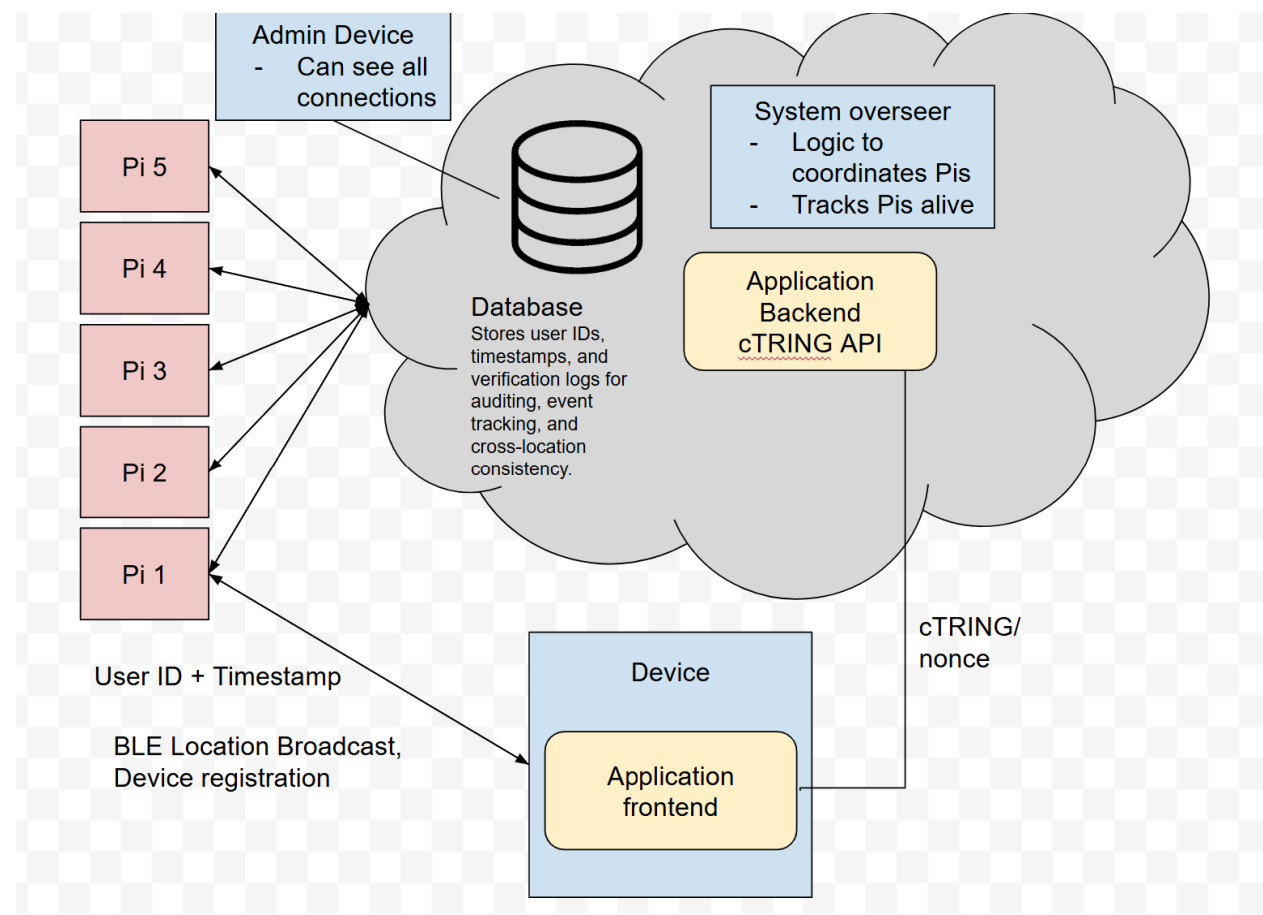
- Secure access tokens valid only within verified physical regions.
- Location-based multiplayer game modes (e.g., capture-the-flag).

This approach delivers new cryptographic primitives for location-aware security, alongside reusable SDKs, APIs, and technical documentation, validating the feasibility of a new class of secure, location-bound applications.

## Assumptions

Our main success metric would be location verification accuracy. Specifically, how often does our system have false positives and negatives? The former could be more of a concern than the latter if our technology was needed to maintain confidentiality. Another key metric would be range of detection - if our Raspberry Pi's struggled to detect users from the other side of a classroom, then users might have to deploy multiple devices which would increase cost. We are assuming that they will have the signal strength needed to accomplish this. Key personas would be application developers and users of said applications - we need to make sure that our system is intuitive enough to develop on and that our example application is user friendly to convince developers to switch to it.

## System Architecture Overview



# Requirements and System Models

## **User Stories**

- User interaction and design (1+ pages) – ie detailed design
- Requirements (functional and non-functional)
- User stories or use cases → 20+ for PRDv2 prioritized w/acc. tests
- Prototyping code, tests, metrics (10+ user stories)
- Each story/case in PRD should include: 1+ links to trello cards, each card should have links to task cards, task cards should have 2+ team members assigned and github commit links for implementations and acceptance tests for the task
- System models (1+ pages)
- Contexts, interactions, structural, behavioral (UML)
- Use cases, sequencing, event response, system state, classes/objects

As a user,

- I want to check in at a location so that I can access features or rewards that are only available on-site.
- I want to get a confirmation message after I check in so that I know my attendance or access was successful.
- I want my check-in data to stay private and secure so that I can trust the system with my location.
- I want to easily find nearby check-in devices so that I can connect without confusion.
- I want to register myself with a Raspberry Pi that allows me to connect with all of the ones on campus.

As an admin,

- I want to set up and manage check-in devices around campus so that users can verify their presence in different locations.
- I want to see activity and records from each device so that I can make sure everything is working properly.
- I want to create and manage events, like exams or raffles, so that only people who check in on-site can participate.

Functional Requirement (needs 5)

- A Raspberry Pi should be able to recognize the hardware identity of a device that is nearby so that it cannot be spoofed

- Cryptographic key generation should be based on the ctRNG (cosmic true random number generator)
- Our final mobile application must validate proximity using Bluetooth signal strength, timestamp, and signed beacon challenge before issuing a presence confirmation.

#### Non-Functional Requirement (needs 5)

- The system should be able to keep track of 100 users
- The backend service should maintain 99.5% uptime with automatic recovery through Docker orchestration and load balancing.
- All proof-of-presence and random number generation events should be cryptographically signed and logged in an immutable audit trail for later verification by third parties.
- All stored data (presence logs, event metadata, random seeds) should include integrity checks via digital signatures or hash chaining to prevent tampering.
- Average presence verification latency must not exceed 2 seconds under normal network conditions, so that we ensure a smooth user experience.

## Appendices

### **Technologies Employed**

#### **Clients**

- Web: React + Web Bluetooth (TypeScript)
- Mobile wrapper: Capacitor

#### **Raspberry Pi**

- Raspberry Pi OS + BlueZ (BLE GATT)
- Runtime: Python or Go
- Crypto: Ed25519 via libsodium/OpenSSL

#### **Backend**

- Node.js (Fastify/Nest/Express) or Go (Echo/Fiber)
- PostgreSQL + Redis
- Object storage (S3/MinIO)
- Spacecomputer Orbitport SDK for randomness/verification

#### **Ops/Security**

- Docker + GitHub Actions
- Nginx/Cloudflare TLS
- Prometheus/Grafana (metrics), Loki (logs)

### **Questions - TODO**

What might the transition from Raspberry Pis to satellites look like? How might we want to design our software to facilitate it? Will our demo application be used to market SpaceComputer?

## **Scope**

A cross-platform mobile app (iOS/Android) and a web console connect to on-site Raspberry Pi devices ("Verifier Nodes"). Users complete a cryptographic presence check. Admins manage sites, devices, campaigns, and audit logs. We plan to make a basic application that is based on this service that also leverages the ctRNG. Note that the satellite software and random number generation is out of scope for this project, as this was done for us by the SpaceComputer team. Also, our Raspberry Pi network is a proof of concept for